

[LAB-1] Introduction to Git & GitHub

Objectives

-  Use IntelliJ IDEA to create the Git Repository and place a Java Project under version control.
-  Learn how to commit changes and other Git operations.
-  Create a GitHub account and keep track of changes in the remote repository.

Software Development Environment

-  Java SE Development Kit **JDK 21** – you need to register an Oracle account:
<https://www.oracle.com/java/technologies/javase/jdk21-archive-downloads.html>
-  **IntelliJ IDEA Community Edition:** <https://www.jetbrains.com/idea/download> (Select Community Edition for download)
-  **GitHub Education Account:** <https://www.github.com>

Guided Lab Exercise&Lab Assignment

-  **Exercise 1:** Create a Java Project in **IntelliJ**;
-  **Exercise 2:** Set up a local **Git** repository for the Java project;
-  **Exercise 3:** Setup a remote Git repository on **GitHub** for collaborative team works.
-  **LAB ASSIGNMENT:** To earn 2 credit points for this lab, you must complete the lab assignment and submit the correct answers (Your GitHub URL) through Canvas by the due date. You should go through all the guided exercises for better prepare the lab assignment.

Exercise 1: Create a Java Project in IntelliJ

Step 1.1 Use Maven to build a new project

1. Launch IntelliJ IDEA.
2. From the main menu, select **Projects | New Project**.
3. We learn to build everything from scratch, so, please **DO NOT** select the checkbox of “Create Git repository” and “Add sample code”.
4. Give a name to the new project: “**Comp3111LEx**”
5. Select “**Java**” language and “**Maven**” as the build system.
6. Choose JDK version 21 you have installed, then click “**Create**” to build the project.
7. You will find below key folders and files under the new project will then be auto-built by IntelliJ.
 - a. [.idea] folder contains the system configuration files.
 - b. [src\main\java] folder contains project source codes you’re going to create.
 - c. [src\test\java] folder contains test source codes.
 - d. [.gitignore] file: You can update .gitignore file
<https://docs.github.com/en/get-started/getting-started-with-git/ignoring-files> to tell Git which files and directories to ignore when you make a commit. More information about .gitignore:
<https://www.jetbrains.com/help/idea/configure-modules.html#module-lang-level> to understand the structure of modules in a Java Project.
 - e. [pom.xml] is an XML file that contains information about the project and configuration details used by Maven to build the project.

Step 1.2 Configuration of JUnit

For our project to use JUnit features on Lab7, we need to add JUnit as a dependency.

1. Right-click the root directory and select “**Open Module Settings**”.
2. From “**Libraries**”, click the “**+**” button and select “**From Maven...**”.
3. Type “**org.junit.jupiter**” and press the **Search** button.
4. Choose the one with the latest version, e.g. “**org.junit.jupiter:junit-jupiter:5.11.0**”, and press “**OK**”.

[LAB-1] Introduction to Git & GitHub

5. Press “OK” to confirm adding the module.
6. Click “Apply” and then “OK” to confirm changes.
7. Double-click to open [pom.xml](#) in the root directory of your “Project” tool window.
8. In [pom.xml](#), right click at any line between `<project>` and `</project>`, and press “Generate”.
9. Choose “Dependency”.
10. In the “Dependencies” tool window that opens, type [org.junit.jupiter:junit-jupiter](#) in the search field. Find the one with the name exactly “[org.junit.jupiter:junit-jupiter:5.x.x](#)”. It is ok if the version is not the same with what we added in *step 4* for now. Click “Add” to add the dependency.
11. Now you should see the dependency added as shown in the *Appendices*. If the version does not match what you have added to the library, change accordingly.
12. Press the [Maven Icon](#) on the right toolbar, [reload the project](#) and you should see the dependencies being correctly added to the project.

Step 1.3 Write application code

Let's start to construct a simple application with class library and main executable java code.

1. Span the `[src]` tree directory and right-click `[src.main.java]`.
2. Select [New | Package](#) and name the package as `[Lab1]`.
3. When `Lab1` package is created under `[src.main.java]`, right-click `[Lab1]` package.
4. Select [New | Java Class](#) and name the Java Class as `[myLibrary]`.
5. Copy example code (from *Appendices*) into `[myLibrary]` java file. There're two functional classes “power” and “factorial” in the `[myLibrary]` java file under `[Lab1]` package.
6. Right-click `[Lab1]` package and select [New | Java Class](#) again, then name the file as `[mainApp1]` to be the main application program. Copy example code (from *Appendices*) into `[mainApp1]` java file.
7. Now you have completed 2 java classes;
8. Click the “Main Menu” on the upper left corner to open the menu bar, and select “[Build Project](#)” under “Build”. The maven system will help to build project resource and compile all source code under the `[Comp3111Ex]` project.
9. If the project is bug-free, you will get relevant results under “Build” tool window (check *Appendices*).
10. If the project is bug-free, you can click the green triangle icon to run the program and get the program's output in the “Run” tool window.
11. Now, we go to make a typo on line 10 code from “else” to “eelse” in `[myLibrary]` java file;
12. Then, go to `[mainApp1]` java file and build project again;
13. The system will prompt error message in “Build” tool/window, and open `[myLibrary]` java file for your action of correction. Indication of the lines where problems came from will be shown on both “Build” tool window and `[myLibrary]` java file.
14. You may keep continue for self-study on “Run” & “Debug” functions under the IntelliJ main menu.

Exercise 2: Set up a local Git repository for the Java project

Step 2.1 Put the existing Java project under VCS

IntelliJ IDEA allows us to quickly enable our project's integration with a version control system (VCS) and associate it with the project root.

1. From the main menu of IntelliJ, select [VCS | Enable Version Control Integration](#).
2. Choose “Git” as the version control system and click “OK”.
3. Wait for a while the entire project will then be associated with a single Git directory, so there's no need to add each file to the Git directory individually.
4. When the local Git repository is built, the “VCS” menu will be changed to “Git” menu afterward.
5. After VCS integration is enabled, IntelliJ IDEA will ask you whether you want to share project settings files via VCS (it may take a few minutes). You should choose “[Always Add](#)” to synchronize project setting with other repository users, like

[LAB-1] Introduction to Git & GitHub

your group project teammates, TAs etc., who work with IntelliJ IDE.

6. Always keep monitoring the status of Git operation: Check the “Show Git Log” function under “Git” tool window. Check Timeline’s actions under the Right Top “Notifications” column.

Step 2.2 Git Operation – Committing changes

Now we’re going to add new files to the Git repository, or modify files that are already under Git VCS. If we’re happy with their current state, we can share the results of our works. This involves committing them locally to record the snapshot of our repository to the project history, and then pushing them to the remote repository so that they become available to others.

1. Open the [.gitignore] file under the root directory.
2. Delete the first line of “target/” to allow built class files to be committed.
3. Select “Commit” from “Git” main menu.
4. Select files from all the [.xml], [.java] and [.class] files in both “Changes” and “Unversioned Files” for the initial commit. The number of [.xml] files can vary for everyone.
5. Give comments to the commit, for instance: “This is an initial commit for Comp3111LEx Project”.
6. Click “Commit” at the left bottom of the “Commit Changes” window.
7. If the commit process is succeeded, the system will prompt a message “8 files committed” on “Git” tool window.
8. We go to modify [mainApp1] java file: add 2 print lines (See Appendices), then select this file and write “Add 2 print lines as report heading” on “Commit Message”, then select “Commit” again under “Git” main menu, we can have the second commit log produced.
9. Select “Log” under “Git” tool window, you will find there’re 2 entries of committed log.

Step 2.3 Git Operation – Push commit(s) to a remote repository

We shall use GitHub as the remote repository for the course project and labs. Here we assume you have already created a GitHub account. **IF NOT**, please **skip this step** and run through Exercise 3 before go ahead to practice below **Push** Git operation.

Why do we need a remote repository?

If you created a Git repository based on local sources, you need to add a remote repository to GitHub to be able to collaborate on your Git project, as well as to eliminate the risks of storing all of your codebase locally. You **PUSH** changes to a remote repository when you need to share your work and **PULL** data from it to integrate changes made by other contributors into your local repository version.

If you have cloned a remote Git repository from GitHub, the remote is configured automatically and you do not have to specify it when you want to sync with it. The default name Git gives to the remote you’ve cloned from is origin.

1. Add your GitHub account in IntelliJ setting: click right-top “IDE Setting” icon or press **Ctrl+Alt+S** to show the IDE Setting, then select “Settings...”.
2. The “IDE Setting” icon will be changed to another shape with grey color once you click it.
3. Select “GitHub” under “Version Control” of “Setting” dialog.
4. Press “+” sign and choose either “Log in via GitHub” or “Log in with Token”.
5. You need to copy your GitHub URL for “Log in via GitHub...” or copy the Token generated by GitHub for “Log in via Token...”.
6. When your GitHub account is listed under **Version Control | GitHub** folder, your remote repository is associated with your IDE project. Press “Apply” and “OK” to save changes.
7. From the main menu, select **Git | GitHub | Share Project on GitHub**, then click “Share” button.
8. The IDE will then push all final commits into your remote GitHub, and, message “Successfully shared project on GitHub” will be shown on Git Console.
9. Now you may check all your push data in your remote repository under your GitHub account.
10. Keep further explore on modify the source code in mainApp1 java file, for instance, add the 3rd print line.
11. From main menu, select **Git | Commit** to commit the code changes.

12. From main menu, select [Git | Push](#), highlight the new commit to update the GitHub. Check all updates in GitHub.

Exercise 3: Set up a remote Git repository on GitHub for collaborative team works

If you have experience in GitHub and already have a GitHub account, you may skip below steps 3.1 to 3.3.

Step 3.1: Introduction to GitHub

GitHub is a web-based code hosting service for version control with collaborative team works. It lets you and others work together on projects of software development from anywhere. If you have zero experience in GitHub, please take a roughly review on Introduction to GitHub: <https://docs.github.com/en/get-started/quickstart/hello-world#introduction>

Step 3.2: Create your personal GitHub account

For COMP3111 lab assignments and the team project, you need to register a GitHub account to host the Java code and project related materials, through this link: <https://www.github.com>. To register a free GitHub account, we suggest you use your HKUST ITSC account (@ust.hk) instead of your HKUST Connect account (@connect.ust.hk) or other external domain email account. For instance, if your HKUST email account is “csalan”, please use csalan@ust.hk instead of csalan@connect.ust.hk.

GitHub Education account is available for HKUST students. Once you have created your GitHub account, you may apply for it with more advanced features for better experience in managing a team project, through this link: <https://education.github.com/students>.

1. Press “Sign up for global campus” button.
2. Use your HKUST ITSC email to sign in GitHub Education. i.e. csalan@ust.hk, NOT csalan@connect.ust.hk! It is because GitHub Education only recognizes HKUST domain “@ust.hk”.
3. Select as a “Student” member.
4. Choose “Free GitHub Pro while you are a student”.
5. When you have seen the full name of HKUST which are displayed automatically, your application is probably succeeded (see Appendices Page 12).

Step 3.3: Generate a GitHub Tokens

There are two purposes of GitHub Personal Access Tokens:



Personal access tokens are an alternative to use passwords for authentication to GitHub Enterprise Server when using the GitHub API or the command line.



Personal access tokens are intended to access GitHub resources on behalf of you.

1. Sign in with your GitHub account. In the upper-right corner, click your profile photo, then select “Settings”.
2. In the left-most sidebar of the prompt page, roll down the scrollbar to locate and select “Developer settings”.
3. Click “Personal access tokens”, then select “Tokens (Classic)”.
4. Click “Generate new token”, then select “Generate new token (Classic) for general use”.
5. Input “For comp3111 Lab1” on “Note” field.
6. Select “custom” on “Expiration” field and change the expiration data for more than 6 months.
7. Tick on the “repo” scope for this token.
8. Click “Generate token” button.
9. Copy the token string to a safe place for later use. Treat your tokens like passwords and keep them secret. You can treat your token as a more secured password with expiration date for your GitHub account with specific granted permissions. You need to do this job immediately because you won’t be able to see it again after you left the page of token generation.
10. Now you have completed the fundamental set up of GitHub account. Please go back to complete step 2.3 – Push commit(s) to your remote GitHub.

LAB ASSIGNMENT:

1. Conduct your own research to **change your lab project for public access** in the GitHub. If you don't make your project available for public, you need to study how to invite TAs by email, otherwise, TA can't check your lab assignment for grading.
2. Conduct your own research on how to **create a readme.md file on the Maven project in IntelliJ**
3. **Writes some comments or remarks** in the readme.md file then **push it to GitHub**.
4. Open this lab project "Comp3111LEx" on IntelliJ. **Make a screenshot** on IntelliJ then find a way to **insert the screenshot into the readme.md file**.
5. **The screenshot should cover** three major tool windows of the Maven project (see Appendices):
 - a) Project tool window with spanning tree of ".idea" and "src".
 - b) Java Class file editor of "myLibrary" or "mainApp1".
 - c) Git | Log tool window with at least three commits listed.
6. Finally, you should get the result like screenshot on Appendices shown on your GitHub.
 - a) Login GitHub and go to the [Comp3111LEx](#) project directory `Comp3111LEx/src/main/java/Lab1`, copy the GitHub URL: `https://github.com/yourGitHubAccount/Comp3111LEx/tree/master/src/main/java/Lab1` and **submit this URL** on Canvas.
 - b) The directory shows **two java files and one readme.md file**. When readme.md file is clicked, readme text contents and the image file should be shown.
7. Hints:
 - a) Allow project public access: Search [GitHub Docs](#) with keywords: [Managing access to your project](#).
 - b) For creating readme.md in Git Repository: Search IntelliJ IDEA Docs | [Non-JVM technologies](#) | [Markdown](#).
 - c) For inserting image in readme.md: Search keywords "[Markdown image](#)".
 - d) To take a screenshot on windows, press Ctrl + PrtScn buttons on your keyboard; To do it on Mac, press Cmd + Shift + 4, the screenshot file will be saved on clipboard.
8. Assessment:
 - a) Successfully established your GitHub Link and submitted on Canvas before due date: Earn full mark of 2.0.
 - b) Deduction:
 - ✓ -1.0 mark would be incurred for missing push of readme.md file;
 - ✓ -0.5 mark would be incurred for missing push of two Java source files;
 - ✓ -0.5 mark would be incurred for failed to view the image file of screenshot of IntelliJ project or failed to provide 3 commits listed in screenshot of IntelliJ project;
 - ✓ -0.5 mark would be incurred for late submission with no more than three days after submission due date.
 - c) Late submission more than three days and without prior approval from lecturer is not acceptable.